flora-publisher 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: "Flora Publisher Factory - Market Research" — 1인 출판사용 SaaS의 첫 모듈로, Google Books 공개 API로 키워드 상위 10권을 검색해 카드로 보여주는 React SPA (frontend/metadata.json:2-3).
- 목적/대상 사용자: 출간을 기획 중인 1인 출판사/작가가 유사 도서를 벤치마킹하는 용도. 사이드바에는 대시보드, 스튜디오, 도서 랜딩, 정산 메뉴가 있으나 모두 "준비 중" 화면이다.
- 출처: Google Vertex AI Studio에서 내려받은 앱 템플릿(README, backend/server.js:2-6 Google LLC 라이선스 헤더) 위에 검색 화면만 얹은 형태다. 저장소에는 커밋 5f6d262 first-pull!! 1개로만 들어와 있고 이후 변경 이력이 없다.

기술 스택

레이어	기술	근거
프론트	React(latest), Vite(latest), TypeScript(latest, tsconfig 없음), Tailwind CDN, lucide-react, 자체 shadcn 풍 UI 4종	frontend/package.json, frontend/index.html:7
백엔드	Node.js Express 5 프록시, google-auth-library(ADC), express-rate-limit, ws	backend/package.json, backend/server.js
DB	없음	-
인프라	로컬 전용 (concurrently로 프론트/백 동시 실행), 배포 설정 없음	package.json:5-8, README.md:6-7
AI	Vertex AI 프록시(템플릿 기본 제공)만 존재, 앱 코드에서 호출하는 곳 없음	backend/server.js:50-120, frontend/components/Settings.tsx:71 "(예정)"

현재 완성도 추정: 약 10%

- 완료: 도서 검색 1개 화면(검색, 로딩 스켈레톤, 빈 결과, 에러 처리)과 설정 화면의 정적 UI.
- 미완: 사이드바 6개 메뉴 중 5개가 미구현(frontend/App.tsx:49-61). 설정 저장, "연결 테스트", "이 도서 벤치마킹하기" 버튼 모두 핸들러가 없다. AI, DB, 인증, 결제는 전혀 연결되지 않았다. 동봉된 PRD(frontend/docs/prd/03_features.md:3,13)는 Next.js App Router 기반 "NOMAD E-Book Factory"를 전제로 하여 현재 Vite SPA 구조와 맞지 않는다.

핵심 기능 목록

- Google Books 키워드 검색(최대 10건) 및 결과 정제(frontend/services/bookApi.ts:11-50)
- 도서 카드 표시(썸네일 https 변환, 저자/출판사/출판일/설명)
- 설정 화면: Google Books 엔드포인트 표시, Vertex AI/Toss Payments 입력란(비기능)
- 템플릿 기본 기능: /api-proxy(Vertex REST), /ws-proxy(Live API WebSocket) 프록시와 브라우저 fetch/WebSocket 가로채기 shim

2. 구조 분석

경로	역할	비고
package.json	npm workspaces + concurrently로 dev 동시 실행	concurrently: latest

경로	역할	비고
backend/server.js (455줄)	Express 프록시: ADC 토큰을 붙여 Vertex AI로 전달, WS 업그레이드 중계	앱 기능에서는 미사용
backend/.env.local	포트, 호스트, GCP 프로젝트/리전, PROXY_HEADER	.gitignore로 제외됨(정상)
frontend/index.tsx	진입점, vertex-ai-proxy-interceptor.js 선 로드	
frontend/vertex-ai-proxy-interceptor.js (154줄)	window.fetch/window.WebSocket 전역 패치	프록시 헤더 값 하드코딩
frontend/App.tsx (176줄)	뷰 전환 상태 + 검색 화면 전체	라우터 없음, 검색 로직이 App에 직접 존재
frontend/services/bookApi.ts	Google Books 호출과 데이터 정제	브라우저에서 직접 호출
frontend/components/	Sidebar, BookCard, Settings, ui(button/card/input/tabs)	모두 소형, 거대 컴포넌트 없음
frontend/lib/config.ts	외부 API URL 상수	Vertex/Toss URL은 표시용
frontend/docs/prd/	유저 플로우, 핵심 기능 명세	다른 제품명/스택 기준 문서

데이터 흐름: 사용자가 키워드를 입력하면 App.handleSearch가 searchBooks를 호출하고, 브라우저가 googleapis.com/books/v1/volumes를 API 키 없이 직접 호출해 결과를 Book[]로 정제한 뒤 카드로 렌더링한다. 백엔드는 이 흐름에 관여하지 않는다. 백엔드는 프론트 shim이 aiplatform.googleapis.com 요청을 가로채 /api-proxy로 보낼 때만 쓰이며, 서버의 ADC 자격 증명으로 토큰을 발급해 Vertex AI에 전달하는 구조다. 따라서 API 키가 프론트 번들에 노출되는 문제는 없다(Google Books는 무키 호출, Vertex는 서버 측 ADC).

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	보안	backend/server.js:104-106, backend/server.js:113-115	ReasoningEngine 패턴의 endpoint_location이 [^/]+로만 매칭되어 evil.com# 같은 값이 호스트 자리에 그대로 들어간다. 조작된 originalUrl로 서버의 Google Cloud 액세스 토큰이 임의 호스트로 전송될 수 있다(SSRF/토큰 유출). project_id도 요청자가 지정 가능
2	높음	보안	frontend/vertex-ai-proxy-interceptor.js:116, backend/server.js:194	유일한 접근 제어인 X-App-Proxy 값이 프론트 소스에 평문 하드코딩되어 번들에 포함된다. 사실상 인증이 없으며, 백엔드가 127.0.0.1 바인딩이라는 점만이 방어선이다
3	중간	보안	backend/server.js:232	클라이언트가 보낸 headers가 서버 헤더 뒤에 펼쳐져 Authorization, X-Goog-User-Project를 덮어쓸 수 있다
4	중간	보안	backend/server.js:388, backend/server.js:216,225	WebSocket 업스트림 메시지 전문과 프록시 URL을 콘솔에 기록한다. 프롬프트/응답 내용이 로그로 남는다
5	중간	설정	backend/server.js:10	node-fetch를 import하지만 backend/package.json에 선언이 없다. 루트 node_modules에 전이 의존성으로 호이스팅되어 우연히 동작 중이며, 의존성 트리 변경 시 서버 기동이 실패한다
6	중간	설정	frontend/package.json:11-21, package.json:10	모든 의존성이 latest로 지정되어 재설치 시 메이저 버전이 바뀔 수 있다(재현 불가 빌드)

#	심각도	분류	위치(경로:줄)	내용
7	중간	버그	backend/server.js:402-416	클라이언트 메시지 JSON 파싱 실패 시 ws.close() 후에도 빈 객체 {}를 업스트림으로 전송한다(return 누락)
8	낮음	버그	frontend/index.html:89	존재하지 않는 /index.js를 로드하는 script 태그가 남아 매 로드마다 404가 발생한다
9	낮음	버그	frontend/App.tsx:21-42, frontend/services/bookApi.ts:18	검색 요청 취소(AbortController)가 없어 연속 검색 시 늦게 온 이전 응답이 최신 결과를 덮어쓸 수 있다. 무키 호출이라 공용 쿼터 초과(429) 시 일반 오류 문구만 표시된다
10	낮음	품질	frontend/index.html:7, frontend/App.tsx:64	Tailwind를 CDN 런타임으로 사용(운영 비권장). animate-in fade-in 클래스는 tailwindcss-animate 플러그인이 없어 동작하지 않는다
11	낮음	품질	frontend/App.tsx:170, frontend/components/Sidebar.tsx:22	고정 폭 사이드바 w-64 + ml-64로 모바일에서 본문이 거의 보이지 않는다(반응형 미지원)
12	낮음	품질	frontend/components/Settings.tsx:39-41,82-84,103-105, frontend/components/BookCard.tsx:58	저장/연결 테스트/벤치마킹 버튼이 동작 없는 더미 UI다. Project ID 입력이 type="password"로 되어 있다
13	낮음	구조	frontend/vite.config.ts:10, backend/server.js:7	사용하지 않는 process.env.API_KEY 더미 define, --env-file과 중복되는 dotenv/config import 등 템플릿 잔재
14	낮음	구조	frontend/docs/prd/03_features.md:3,13, frontend/services/bookApi.ts:4-9	PRD는 Next.js 라우트(app/api/vertex/write/route.ts)와 다른 제품명을 전제로 하며, 코드 주석 스스로 PRD 구조를 따르지 못했다고 밝힌다
15	낮음	설정	frontend/ (tsconfig 부재), 저장소 전체	tsconfig.json, 린트, 테스트가 없어 타입 검사가 수행되지 않는다(Vite는 트랜스파일만 함)

심각도 높음 상세

1. 토큰 유출 가능 프록시 (#1): 프론트 shim은 호스트가 -aiplatform.googleapis.com으로 끝나는지 검사하지만, 백엔드는 문자열 정규식만으로 매칭한 뒤 https://${endpoint_location}-aiplatform.clients6.google.com/...로 URL을 재조립한다. /api-proxy를 직접 호출해 endpoint_location에 attacker.example# 형태를 넣으면 실제 요청 호스트가 공격자 서버가 되고 Authorization: Bearer <ADC 토큰>이 그대로 전송된다. 현재는 로컬 전용이라 위험이 잠재 상태지만, API_BACKEND_HOST를 바꾸거나 배포하는 순간 GCP 계정 전체 권한(cloud-platform 스코프) 토큰이 유출될 수 있다. endpoint_location, location_id를 허용 리전 목록으로 검증하고 project_id는 환경 변수 값으로 고정해야 한다.
2. 하드코딩된 프록시 헤더 (#2): PROXY_HEADER는 "요청이 shim에서 왔는지"를 확인하는 용도인데, 그 값이 프론트 소스에 그대로 들어 있어 번들을 열면 누구나 알 수 있다. #1과 결합하면 백엔드에 도달 가능한 누구나 토큰 유출 요청을 만들 수 있다. 실제 서비스로 갈 경우 이 프록시 대신 사용자 인증(세션/JWT)과 서버 측 고정 엔드포인트(예: /api/books/analyze)로 교체해야 한다.

4. 효율성 개선 제안

- 구조/아키텍처: 범용 Vertex 프록시와 전역 fetch/WebSocket 몽키패치는 현재 쓰이지 않으므로 제거하고, AI 기능이 필요하면 목적별 서버 라우트 1~2개로 대체한다. 뷰 전환은 useState 대신 라우터로 바꾸고 검색 상태/로직은 useBookSearch 훅으로 분리한다.
- 성능: 검색에 AbortController와 간단한 결과 캐시(키워드별 메모리 캐시)를 추가한다. Tailwind CDN 대신 빌드 타임 Tailwind로 전환해 런타임 CSS 생성 비용을 없앤다.
- 개발 생산성: 의존성 버전을 고정하고 node-fetch를 제거(Node 18+ 내장 fetch 사용)한다. tsconfig.json(strict)과 ESLint를 추가하고, bookApi의 데이터 정제 함수에 단위 테스트를 둔다.

- 운영/배포: 현재 배포 경로가 없다. 독립 앱으로 운영하지 않는다면 배포 설정을 만들 필요가 없으며, Flora 계열 본선(아래 6장)에 기능으로 흡수하는 편이 운영 비용이 가장 낮다.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	server.js의 ReasoningEngine 핸들러 2개에서 endpoint_location/location_id를 허용 리전 목록으로 검증하고 project_id를 GOOGLE_CLOUD_PROJECT로 고정, 클라이언트 headers 중 authorization/x-goog-user-project 제거	P0	S	Opus 5.5	범위는 작지만 토큰 유출과 직결되는 보안 수정이라 Sonnet보다 한 단계 위를 쓴다
2	앱이 Vertex를 쓰지 않으므로 /api-proxy, /ws-proxy, vertex-ai-proxy-interceptor.js, index.tsx의 import, vite proxy 설정을 일괄 삭제(또는 #1 적용 후 유지 여부 결정)	P0	S	Sonnet 5.5	사용처가 없는 코드 삭제로 판단 기준이 명확하다
3	프록시를 유지할 경우 PROXY_HEADER 하드코딩을 제거하고 세션 기반 인증 미들웨어로 대체	P1	M	Fable 5.1	인증 설계 판단이 필요한 보안 로직
4	모든 latest 의존성을 현재 설치된 버전으로 고정, node-fetch import 삭제 후 내장 fetch 사용, dotenv 중복 제거	P1	S	Sonnet 5.5	설정 파일 정리
5	index.html:89의 /index.js 태그, vite.config.ts의 API_KEY 더미 define 삭제	P1	S	Sonnet 5.5	단순 정리
6	WS 메시지 파싱 실패 시 return 추가, 업스트림 메시지 본문 로깅을 디버그 플래그 뒤로 이동	P1	S	Sonnet 5.5	단순 버그 및 로그 정리
7	검색 로직을 useBookSearch 훅으로 분리하고 AbortController로 이전 요청 취소, 429 응답 시 별도 안내 문구 표시	P1	S	Opus 5.5	비동기 경쟁 상태 처리가 포함된 상태 관리 정리
8	Tailwind CDN을 빌드 타임 Tailwind(+ tailwindcss-animate)로 전환하고 index.html 인라인 테마를 CSS 파일로 이동	P2	S	Sonnet 5.5	설정/스타일 작업
9	사이드바를 md 이상에서만 고정하고 모바일은 드로어로 전환, main의 ml-64를 md:ml-64로 변경	P2	S	Sonnet 5.5	UI 수정
10	tsconfig.json(strict) 추가, bookApi의 any 제거(Google Books 응답 타입 정의), ESLint 설정	P2	S	Sonnet 5.5	타입/린트 정리
11	더미 버튼(설정 저장, 연결 테스트, 벤치마킹하기)을 숨기거나 "준비 중" 비활성 상태로 표시	P2	S	Sonnet 5.5	UI 문구/상태 수정
12	아카이브 결정 시: bookApi.ts 정제 로직과 BookCard를 Flora-v2 프론트의 리서치 유틸로 이식할 가치가 있는지 판단해 이식 또는 폐기 후 폴더를 archive/로 이동	P1	S	Opus 5.5	다른 저장소 구조를 이해하고 2~3개 파일을 옮기는 중간 규모 작업

6. 종합 평가

- 강점
  1. 코드가 작고 깔끔하다. 검색 화면은 로딩/빈 결과/에러 상태를 모두 갖췄고 데이터 정제가 서비스 계층으로 분리되어 있다.
  2. API 키가 프론트에 노출되지 않는다. Google Books는 무키 공개 호출이고, Vertex 자격 증명은 서버 ADC로만 처리된다. .env.local은 git에서 제외되어 있다.

3. 템플릿 백엔드에 기본 레이트 리밋(15분당 100회)과 localhost 바인딩이 적용되어 있다.

- 약점
  1. 템플릿에서 딸려 온 범용 Vertex 프록시가 미사용 상태로 남아 있고, 토큰 유출 가능 경로(#1)와 하드코딩된 접근 헤더(#2)를 포함한다.
  2. 기능 완성도가 매우 낮다(메뉴 6개 중 1개). PRD와 실제 구조(Next.js 대 Vite SPA, 제품명)가 어긋난다.
  3. 의존성 `latest` 지정, 미선언 의존성, tsconfig/테스트 부재로 재현성과 타입 안정성이 없다.
- 모델별 비율 (12개 작업 기준): Fable 8% (1개) / Opus 25% (3개) / Sonnet 67% (8개)
- 권장 진행 순서
  1. 1단계: 보안 위험 제거(#1 또는 #2 중 택일 — 미사용이므로 #2 삭제가 더 단순), 의존성 고정(#4), 잔재 정리(#5, #6).
  2. 2단계: 프로젝트 방향 결정. 아래 권고대로 아카이브한다면 #12만 수행하고 종료한다.
  3. 3단계: 계속 발전시키는 경우에만 #7~#11 및 #3을 수행한다.

Flora 계열과의 관계 및 권고: 아카이브

구조 수준 비교 결과, `flora`(Next.js 16 + FastAPI + Supabase, 소스 파일 다수)와 `Flora-v2`(Next.js + FastAPI `api/routers·services` + Supabase, 약 94개 소스 파일, PRD/DESIGN/SKILL 문서 보유)는 둘 다 "1인 출판 AI SaaS"라는 동일한 제품을 훨씬 넓은 범위로 구현하고 있다. flora-publisher의 사이드바 메뉴(대시보드, 스튜디오, 정산)와 동봉 PRD(자동 집필 큐, 소셜 로그인)는 이 두 프로젝트의 부분집합이며, 독자적인 부분은 Google Books 검색 화면 하나뿐이다. 또한 flora의 LLM 라우터는 "벤치마킹 방식 폐기"를 명시하고 있어(`flora/api/routers/llm.py:846, :1118`) 계열 내에서 이 모듈의 방향성 자체가 이미 기각된 상태다. 기존 리뷰(`docs/reviews/flora.md` 6장)가 Flora-v2를 본선으로 권고한 것과도 일관되게, flora-publisher는 "계속 발전"이나 "통합"할 만큼의 자산이 없으므로 아카이브를 권고한다. 다만 시장 조사 기능을 다시 살릴 계획이 있다면 `services/bookApi.ts`(50줄)와 `BookCard.tsx`(65줄)만 Flora-v2로 옮기면 충분하며, 백엔드 프록시는 이식하지 않는다.

주 담당 모델: Sonnet 5.5

작업 12개 중 8개가 의존성 고정, 잔재 삭제, UI/설정 정리처럼 판단 기준이 명확한 소규모 작업이고, 프로젝트 자체가 아카이브 대상이라 대규모 설계 변경이 필요 없다. 보안 수정(#1)과 이식 판단(#12), 검색 훅 정리(#7)만 Opus 5.5에 맡기고, 프록시를 계속 운영하기로 결정한 경우에 한해 인증 설계(#3)를 Fable 5.1에 배정하면 된다. 전체 비용 대비 효과를 고려하면 Sonnet 5.5가 주 담당으로 가장 적합하다.